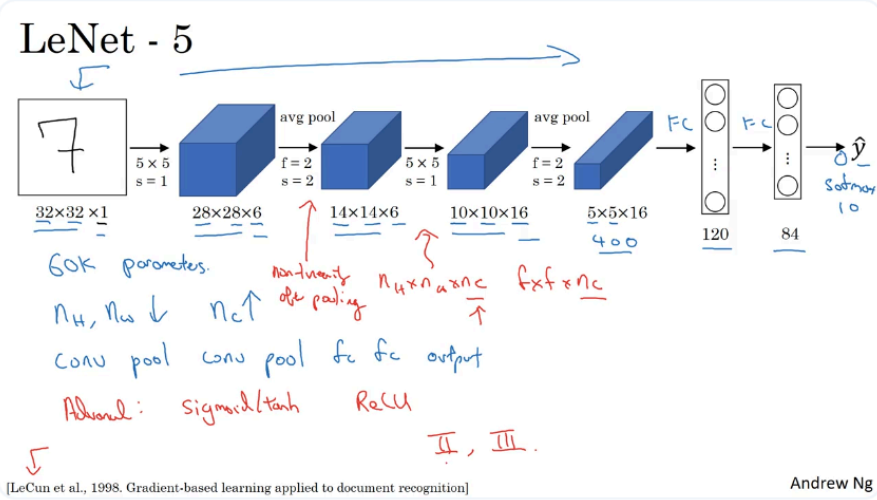


Case Studies

Classic Networks

LeNet - 5



LeNet-5 Architecture (1998)

Goal: Recognition of handwritten digits (MNIST dataset).

Key Characteristics: * Designed for 32x32x1 (grayscale) images.

- Total Parameters: ~60,000 (small by modern standards).
- Follows the classic pattern: **[Conv → Pool] x 2 → Fully Connected (FC) → Output**.

Layer-by-Layer Breakdown

Layer Type	Input Dim	Filter/Stride	Output Dim	Notes
Input	32×32×1	-	-	Grayscale digit image
Conv 1	32×32×1	(6) 5×5 filters, s=1	28×28×6	No padding (Valid convolution)
Pool 1	28×28×6	2×2 filter, s=2	14×14×6	Traditionally Average Pooling
Conv 2	14×14×6	(16) 5×5 filters, s=1	10×10×16	No padding
Pool 2	10×10×16	2×2 filter, s=2	5×5×16	Dimensions reduced by half
Fully Conn 1	400 nodes	-	120	5 × 5 × 16 = 400
Fully Conn 2	120	-	84	Intermediate FC layer
Output	84	-	10	Softmax (modern) for digits 0-9

Structural Trends

As you go **deeper** (left to right) into the network:

- **Height & Width (\$n_H, n_W\$):** Tend to decrease.
- **Channels (\$n_C\$):** Tend to increase (from 1 to 6 to 16).

Historical vs. Modern Differences

The original 1998 paper contains several details that are rarely used in modern Deep Learning:

- **Pooling:** Used **Average Pooling** instead of the now-standard **Max Pooling**.
- **Activation Functions:** Used **Sigmoid** or **Tanh**; modern versions almost exclusively use **ReLU**.
- **Layer Connections:** The original paper used a complex "partial connectivity" where filters didn't look at all input channels (done to save computation). Modern versions look at all channels.
- **Non-linearity Placement:** Original LeNet-5 applied a non-linearity *after* the pooling layer.

- **Output Layer:** Used a specific classifier that has since been replaced by the **Softmax** layer for 10-way classification.

Reading Tips

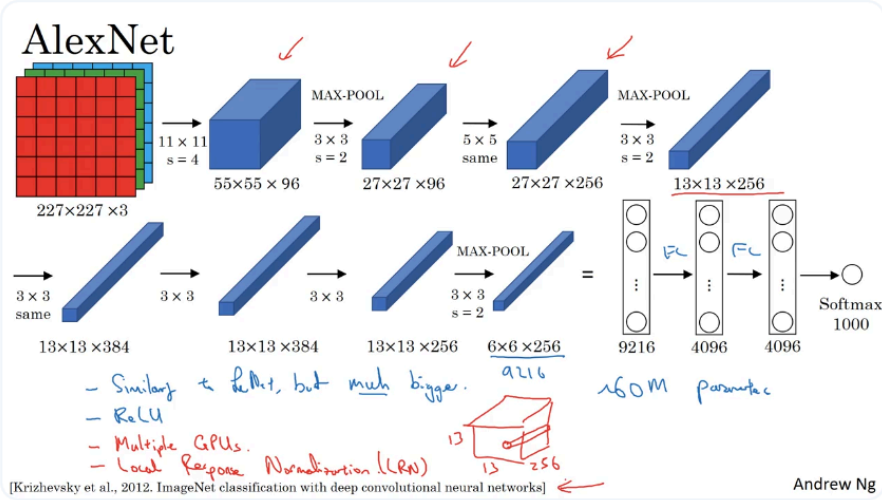
If reading the classic paper by Yann LeCun et al.:

- **Focus on Section 2:** Covers the architecture details.
- **Section 3:** Contains interesting experimental results.
- **Skip:** "Graph Transformer Networks" sections, as they are not widely used in modern computer vision.

[!TIP]

Key Takeaway: LeNet-5 established the "Conv-Pool-Conv-Pool-FC-FC" template that defined Convolutional Neural Networks for decades.

AlexNet



Named after **Alex Krizhevsky** (with Ilya Sutskever and Geoffrey Hinton), this architecture was a pivotal milestone that convinced the computer vision community that Deep Learning actually works.

🔧 Architecture Breakdown

While similar in structure to LeNet, AlexNet is significantly larger, boasting **60 million parameters** (compared to LeNet's 60,000).

Layer	Input Size	Filter/Stride	Output Size	Notes
Input	227×227×3	-	-	RGB Images
Conv 1	227×227×3	11×11, s=4	55×55×96	Large stride causes sharp reduction
Max Pool 1	55×55×96	3×3, s=2	27×27×96	Overlapping pooling (f=3, s=2)
Conv 2	27×27×96	5×5, padding=Same	27×27×256	-
Max Pool 2	27×27×256	3×3, s=2	13×13×256	-
Conv 3	13×13×256	3×3, padding=Same	13×13×384	-
Conv 4	13×13×384	3×3, padding=Same	13×13×384	-
Conv 5	13×13×384	3×3, padding=Same	13×13×256	-
Max Pool 3	13×13×256	3×3, s=2	6×6×256	Final spatial reduction
Flatten	-	-	9,216	$6 \times 6 \times 256$
FC 1	9,216	-	4,096	Fully connected
FC 2	4,096	-	4,096	Fully connected
Softmax	4,096	-	1,000	For ImageNet classes

Key Improvements over LeNet

- **ReLU Activation:** Used ReLU instead of Sigmoid/Tanh, which significantly accelerated training.
- **Scale:** Massive increase in hidden units and depth.
- **Data:** Trained on the **ImageNet** dataset (millions of images).

- **Multi-GPU Training:** Because GPUs were slower in 2012, the original model was split across two GPUs with specific communication points.

Historical Footnotes

[!IMPORTANT]

Local Response Normalization (LRN): > The original paper used LRN, which looks at one position across all channels and normalizes the activations. The goal was to prevent a single neuron from "dominating" its neighbors.

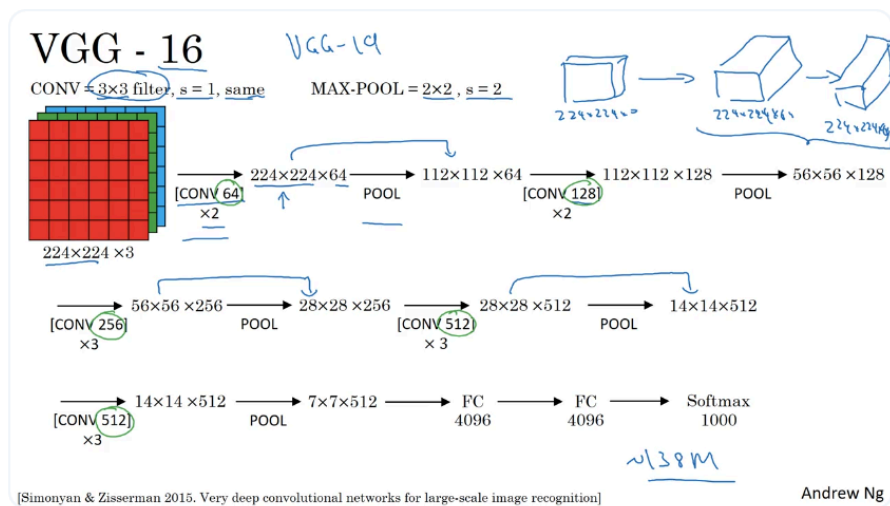
Modern Verdict: Most researchers found this doesn't help much; it is rarely used today.

Why it Matters

- **The Turning Point:** This paper is widely credited with starting the modern "Deep Learning Revolution" in computer vision.
- **Readability:** Compared to other classic papers, the AlexNet paper is considered relatively easy to read for beginners.

How is your Notion page coming along? Would you like a summary comparison of the main differences between LeNet and AlexNet to add to your workspace?

VGG - 16



Named after the **Visual Geometry Group** at Oxford, VGG-16 is celebrated for its simplicity and uniformity. While AlexNet had many hyperparameter choices, VGG focused on a very systematic design principle.

The "VGG Principle"

The beauty of VGG-16 lies in its consistent rules:

- **All Convolutions:** 3×3 filters, Stride (s) = 1, Padding = **Same**.
- **All Max Pooling:** 2×2 filters, Stride (s) = 2.
- **Channel Pattern:** Every time a pooling layer halves the height and width, the number of filters/channels **doubles** ($64 \rightarrow 128 \rightarrow 256 \rightarrow 512$).

Architecture Breakdown

The "16" in VGG-16 refers to the **16 layers with weights** (Convolutional and Fully Connected layers).

Stage	Layer Type	Output Dimension	Notes
Input	Image	224x224x3	RGB Image
Block 1	Conv (64) x2	224x224x64	"Same" padding keeps dimensions
	Max Pool	112x112x64	Height/Width halved
Block 2	Conv (128) x2	112x112x128	Filters double to 128
	Max Pool	56x56x128	Height/Width halved
Block 3	Conv (256) x3	56x56x256	Three conv layers in this block
	Max Pool	28x28x256	Height/Width halved

Block 4	Conv (512) x3	28×28×512	-
	Max Pool	14×14×512	-
Block 5	Conv (512) x3	14×14×512	Stays at 512 filters
	Max Pool	7×7×512	Final spatial dimension
FC	Flattened	25,088 nodes	$7 \times 7 \times 512$
FC	Dense	4,096	-
FC	Dense	4,096	-
Output	Softmax	1,000	ImageNet classes

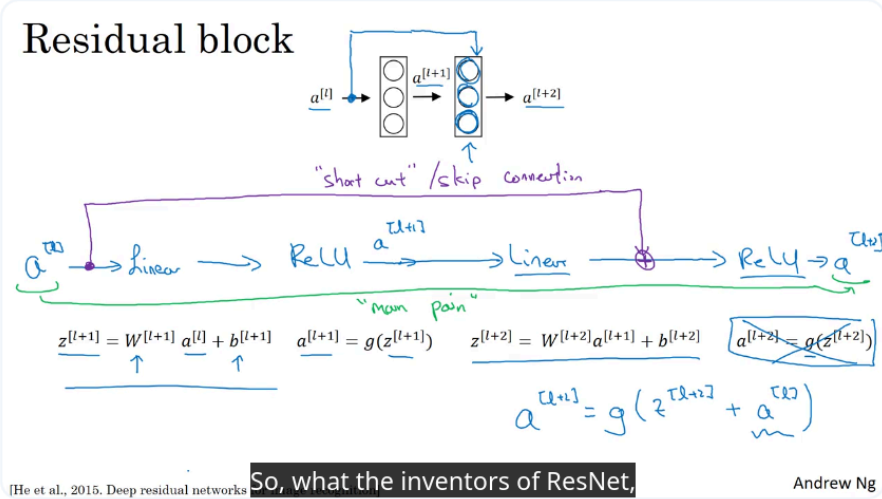
Key Insights

- **Massive Scale:** VGG-16 has about **138 million parameters**. Even by modern standards, it is considered a very "heavy" network.
- **VGG-16 vs. VGG-19:** VGG-19 is an even deeper version, but because VGG-16 performs almost as well with fewer parameters, it is often the more popular choice for transfer learning.
- **Systematic Growth:** The architecture is attractive to researchers because the height/width decrease and the channel increase are incredibly predictable and systematic.

Summary Comparison: Classic Architectures

Feature	LeNet-5	AlexNet	VGG-16
Year	1998	2012	2014
Parameters	~60k	~60M	~138M
Activations	Sigmoid/Tanh	ReLU	ReLU
Pooling	Average	Max	Max
Key Strength	First successful CNN	Scaled up DL for CV	Simplicity & Uniformity

ResNets



ResNet (Residual Networks)

Training very deep networks is challenging due to the **vanishing and exploding gradient** problems. ResNet, introduced by Kaiming He et al., solves this by using **skip connections** (or shortcuts) that allow gradients to flow through the network more effectively.

The Residual Block

In a standard "Plain" network, information must flow linearly through every layer. In a **Residual Block**, we add a "shortcut" that bypasses one or more layers.

1. The Main Path

The standard flow of computation across two layers:

1. **Linear Step 1:** $z^{[l+1]} = W^{[l+1]}a^{[l]} + b^{[l+1]}$
2. **Activation 1:** $a^{[l+1]} = g(z^{[l+1]})$ (using ReLU)
3. **Linear Step 2:** $z^{[l+2]} = W^{[l+2]}a^{[l+1]} + b^{[l+2]}$
4. **Activation 2:** $a^{[l+2]} = g(z^{[l+2]})$

2. The Shortcut (Skip Connection)

In a ResNet, we take the activation $a^{[l]}$ and "feed" it deeper into the network, adding it just before the second ReLU nonlinearity.

The modified equation for the output becomes:

$$a^{[l+2]} = g(z^{[l+2]} + a^{[l]})$$

[!IMPORTANT]

The shortcut is added **after** the linear computation ($z^{[l+2]}$) but **before** the activation function (g).

Plain vs. ResNet

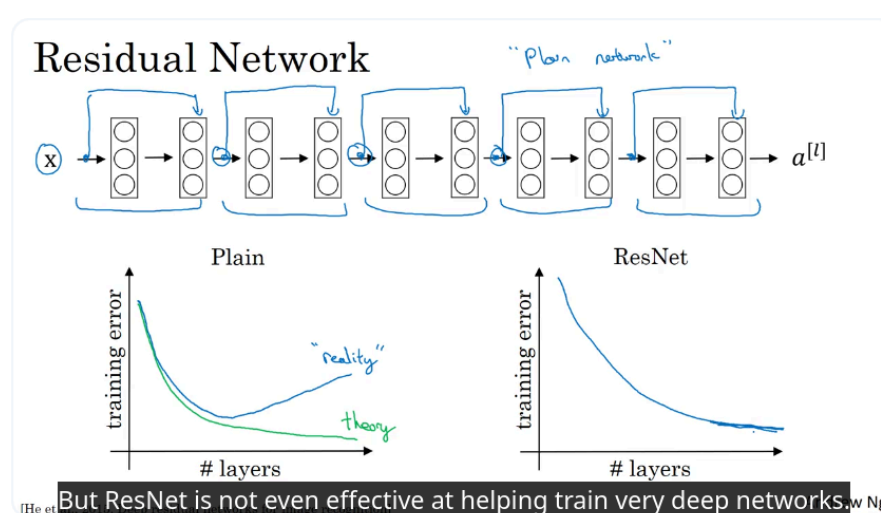
- **Plain Network:** As the number of layers increases, the training error often tends to go down but then eventually starts increasing again (not due to overfitting, but because the network becomes too difficult to train).
- **ResNet:** Even as the network depth increases (sometimes over 100 layers), the training error continues to decrease. The skip connections make it easier for the network to learn the **identity function**, ensuring that performance doesn't degrade with depth.

Key Terminology

- **Residual Block:** The basic building block of a ResNet consisting of the main path and a shortcut.
- **Skip Connection / Shortcut:** The path that allows $a^{[l]}$ to bypass layers and be added deeper in the network.
- **Identity Mapping:** The ability of the network to pass information through the shortcut without changing it, which helps prevent performance loss in very deep architectures.

Historical Context

- **Authors:** Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun.
- **Impact:** This architecture allowed for the successful training of networks with 100+ layers, vastly outperforming previous "shallow" models.



ResNet: Training Performance vs. Depth

A "Plain" network (a standard stacking of layers without shortcuts) suffers from optimization difficulties as it gets deeper. ResNets fundamentally change the relationship between network depth and training success.

The Optimization Gap

In theory, a deeper network should **never** perform worse than a shallower one on training data, because the deeper layers could simply learn the **identity function** (passing the input through without change). However, in practice, standard gradient descent struggles with deeper "plain" architectures.

Plain Networks

- **The Problem:** As layers are added, training error initially decreases but eventually **increases**.
- **Reason:** This isn't overfitting (which would show low training error but high test error). Instead, it is an **optimization problem**—the gradients become so small or unstable that the algorithm cannot find the optimal weights.

Residual Networks (ResNets)

- **The Performance:** Even with 100+ layers, the training error continues to trend **downward**.
- **The Secret:** Because of skip connections, it is very easy for the network to learn "Identity Mappings." If a specific layer isn't helpful, the shortcut allows the network to simply "skip" it, ensuring that adding layers doesn't hurt performance.

Key Benefits of Deep ResNets

- **Vanishing/Exploding Gradients:** Skip connections provide a "highway" for the gradient to flow directly back to earlier layers during backpropagation.
- **Extreme Depth:** Enables the training of networks with 100 to 1,000 layers, though 50, 101, and 152 are the most common practical variations.
- **Plateauing:** While ResNets prevent performance from *worsening* with depth, performance will eventually plateau (flatten out) once the network has sufficient capacity for the task.

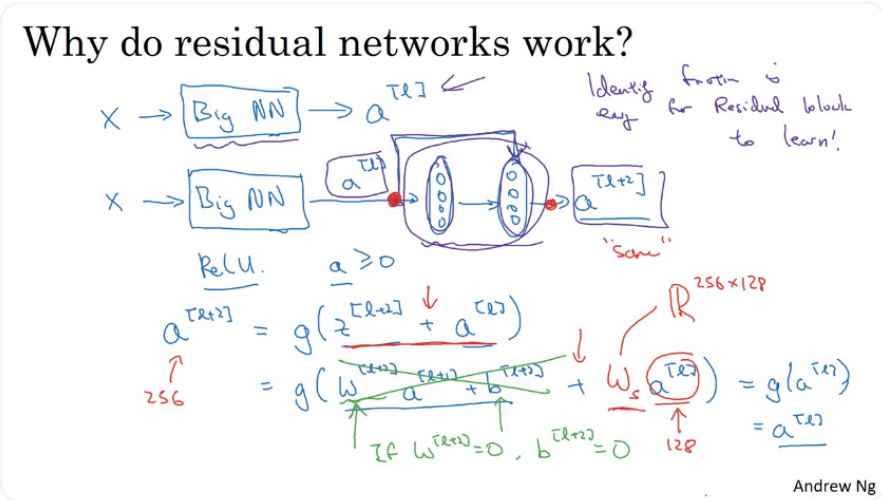
💡 Summary Table: Training Behavior

Feature	Plain Network	ResNet
Ideal Theory	Deeper = Better	Deeper = Better
Empirical Reality	Training error increases after a certain depth.	Training error continues to decrease with depth.
Main Hurdle	Vanishing/Exploding Gradients.	Eventually reaches a performance plateau.
Max Practical Depth	Usually < 20-30 layers.	100+ (even up to 1,000) layers.

[!TIP]

When implementing this in Notion, consider adding a **Callout block** highlighting that ResNet doesn't just "help" training—it's often the only reason networks of that depth are even possible to train.

Why ResNets Work?



Why ResNets Work: The "Identity" Shortcut

The primary reason ResNets are so effective is that they make it **mathematically easy** for a deep network to learn the **identity function**. In a standard network, it is surprisingly difficult for multiple layers to learn to simply "pass through" an input without distorting it.



The Mathematics of the Identity

Consider a Residual Block where the input is $a^{[l]}$ and the output is $a^{[l+2]}$.

$$a^{[l+2]} = g(z^{[l+2]} + a^{[l]})$$

If we apply L_2 **Regularization** (weight decay), the weights $W^{[l+2]}$ and biases $b^{[l+2]}$ will tend to shrink toward zero. Let's look at what happens if $W^{[l+2]} = 0$ and $b^{[l+2]} = 0$:

1. The linear term $z^{[l+2]} = W^{[l+2]}a^{[l+1]} + b^{[l+2]}$ becomes **0**.
2. The equation simplifies to: $a^{[l+2]} = g(0 + a^{[l]})$.
3. Since we use **ReLU = Max(x, 0)**
4. Therefore, $a^{[l+2]} = a^{[l]}$

Conclusion: Even if the weight layers in the block learn nothing (or are useless), the skip connection ensures the network can just "copy" $a^{[l]}$ to the next stage. This guarantees that adding layers **will not hurt performance**, providing a safe baseline for the optimization algorithm to improve upon.



Dimension Matching

For the addition $z^{[l+2]} + a^{[l]}$ to work, the vectors must have the **same dimensions**.

1. "Same" Convolutions

ResNets frequently use **Same Padding** convolutions to ensure the output dimensions match the input dimensions, making the addition straightforward.

2. The W_s Adjustment (Dimension Mismatch)

If the dimensions do change (e.g., the number of channels doubles), we introduce a matrix W_s to the shortcut path:

$$a^{[l+2]} = g(z^{[l+2]} + W_s \cdot a^{[l]})$$

- W_s is a matrix used to resize $a^{[l]}$ to match the dimensions of $z^{[l+2]}$.
- **Implementation:** W_s can be a set of **learnable parameters** or a **fixed matrix** that performs zero-padding to increase the channel count.

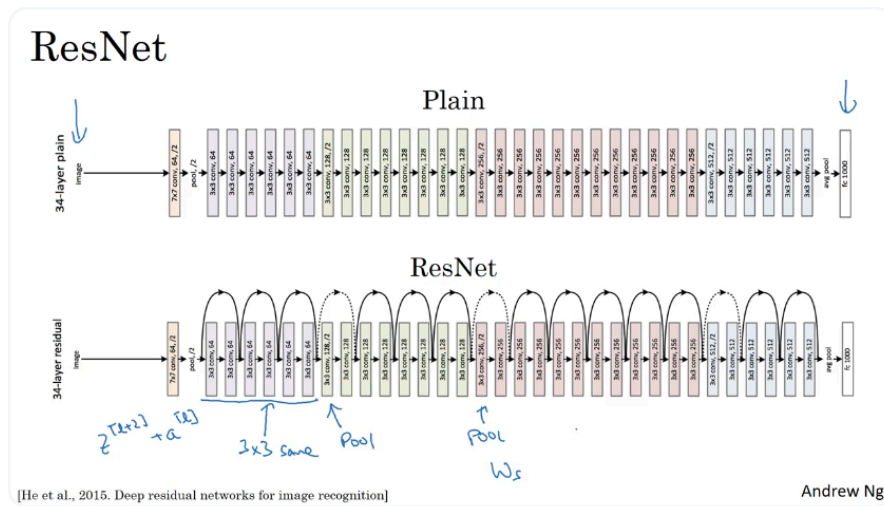


Key Takeaways for Notion

- **Safety Net:** The skip connection acts as a safety net that prevents deep networks from performing worse than shallow ones.
- **Optimization:** It is easier for Gradient Descent to learn that $W = 0$ than to find a complex set of weights that perfectly mimic an identity function in a plain network.
- **ReLU Benefit:** Because ReLU's output is non-negative, $g(a^{[l]})$ stays $a^{[l]}$, facilitating that identity mapping.

[!NOTE]

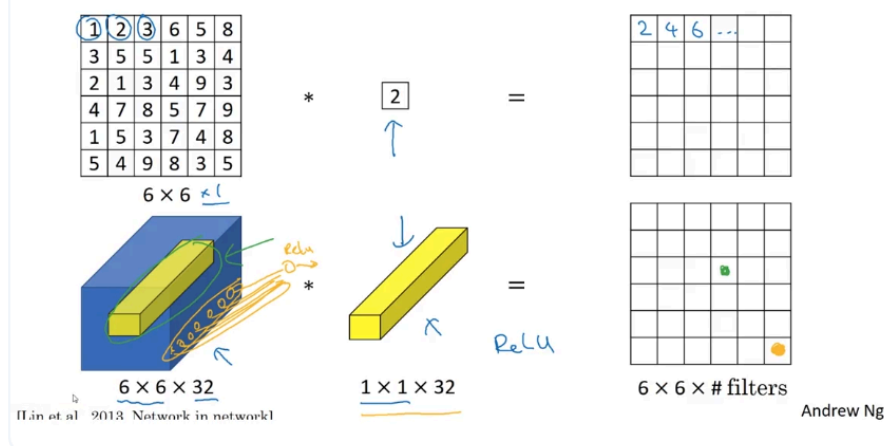
Performance Logic: The goal isn't just to "not hurt" performance; it's to give the network a chance to learn something useful. If the hidden units learn something better than the identity, performance improves. If they don't, the shortcut ensures they don't cause harm.



Andrew Ng

Networks in Networks and 1x1 Convolutions

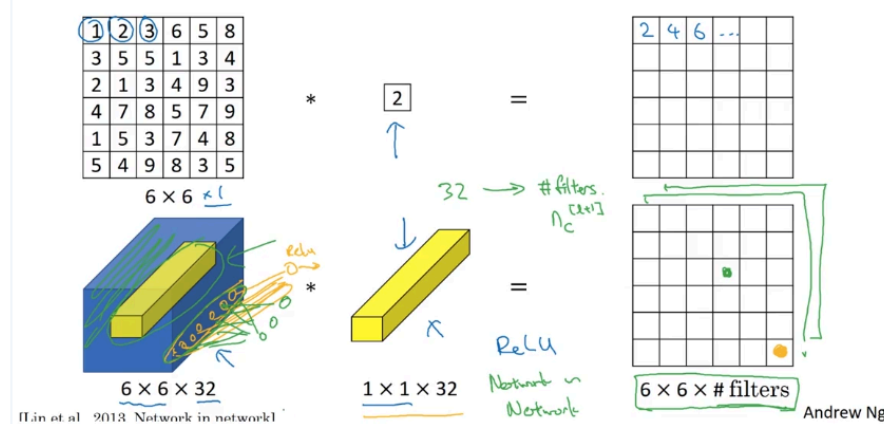
Why does a 1×1 convolution do?



Andrew Ng

Look at the orange color

Why does a 1×1 convolution do?



Andrew Ng

Look at the green color

1x1 Convolutions (Network in Network)

At first glance, a 1×1 convolution seems like simply multiplying an image by a single number. However, when applied to multi-channel volumes, it becomes a powerful tool for **dimensionality reduction and non-trivial computation**.

What does a 1x1 Convolution actually do?

1. Single Channel Case

If you have a $6 \times 6 \times 1$ image and apply a $1 \times 1 \times 1$ filter (e.g., the number 2), every pixel is simply multiplied by that number. This is essentially just **scaling** the image.

2. Multi-Channel Case (The "Neuron" View)

If you have a $6 \times 6 \times 32$ volume and apply a $1 \times 1 \times 32$ filter:

- The filter looks at **one spatial position** (one pixel location) and considers all **32 numbers** (channels) at that spot.

- It performs an element-wise product between the 32 input numbers and the 32 weights in the filter, sums them up, and applies a **ReLU nonlinearity**.
- It repeats this for all 36 positions (6×6).

The "Network in Network" Perspective

A 1×1 convolution is effectively a **Fully Connected Neural Network** applied independently to every single pixel position.

- **Input:** n_C numbers (the channels at one pixel).
- **Output:** A new number of channels based on how many filters you use.
- **Function:** It allows the network to learn complex interactions between channels without changing the spatial (H x W) dimensions.

Why is this useful?

1. **Dimensionality Reduction (Channel Pooling):** You can shrink the number of channels to save computation. For example, you can take a $28 \times 28 \times 192$ volume and convolve it with 32 filters of size 1×1 to get a $28 \times 28 \times 32$ volume.
2. **Adding Non-linearity:** By applying a ReLU after the 1×1 convolution, you add a layer of learning and complexity without significantly increasing the computational cost.
3. **Feature Transformation:** It allows the network to "mix" information across channels in a way that standard pooling cannot.

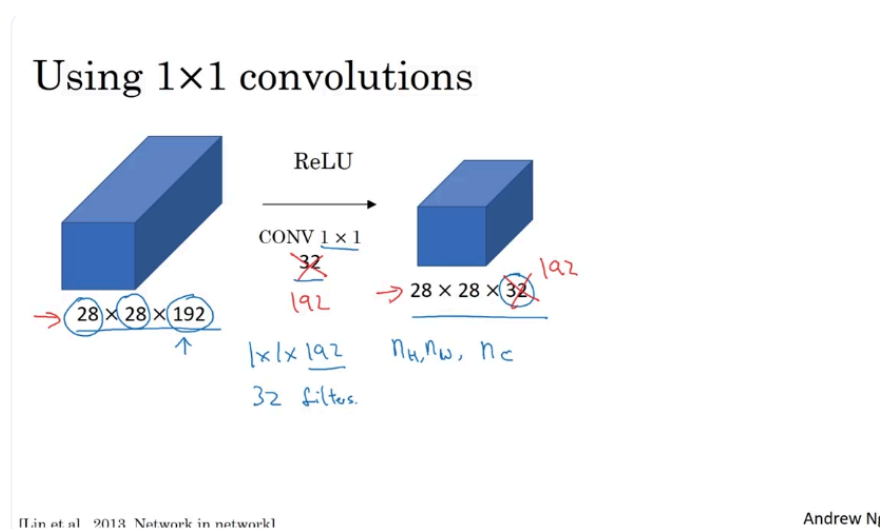
Historical Context

- **Other Names:** Often called **Network in Network (NiN)** based on the influential paper by Lin et al.
- **Impact:** This concept is a fundamental building block for modern architectures like the **Inception Network** and **ResNets**.

[!TIP]

Think of a 1×1 convolution as a way to **pool across channels** rather than pooling across spatial pixels.

Next Step in your notes: You might want to link this to the **Inception Module**, which relies heavily on 1×1 convolutions to manage computational "bottlenecks." Would you like me to summarize how Inception uses these?



Practical Use: The 1×1 Convolution as a "Channel Shrinker"

While **Pooling Layers** (Max/Average) are used to reduce the spatial dimensions (H x W), the **1×1 Convolution** is the primary tool for managing the depth or number of channels n_C .



Shrinking the Channel Dimension

Imagine you have a volume that is $28 \times 28 \times 192$, but you find the 192 channels too computationally expensive for the next layer.

- **The Goal:** Reduce the volume to $28 \times 28 \times 32$.

- **The Solution:** Apply **32 filters** of size **1×1×192**.
- **The Result:** Each filter "collapses" the information from the 192 input channels into a single output channel, resulting in 32 new feature maps.

 Three Main Applications

Use Case	Transformation	Benefit
Shrinking	192 channels → 32 channels	Reduces computational load (Bottleneck layer).
Increasing	32 channels → 128 channels	Adds capacity to represent more complex features.
Keeping Same	192 channels → 192 channels	Adds a non-linear ReLU layer to learn more complex patterns without changing volume size.

 Key Comparison for Notion

It is helpful to categorize your dimensionality reduction tools clearly:

- **Max/Average Pooling:** Reduces **Height** and **Width**.
- **1×1 Convolution:** Reduces (or increases) the number of **Channels**.

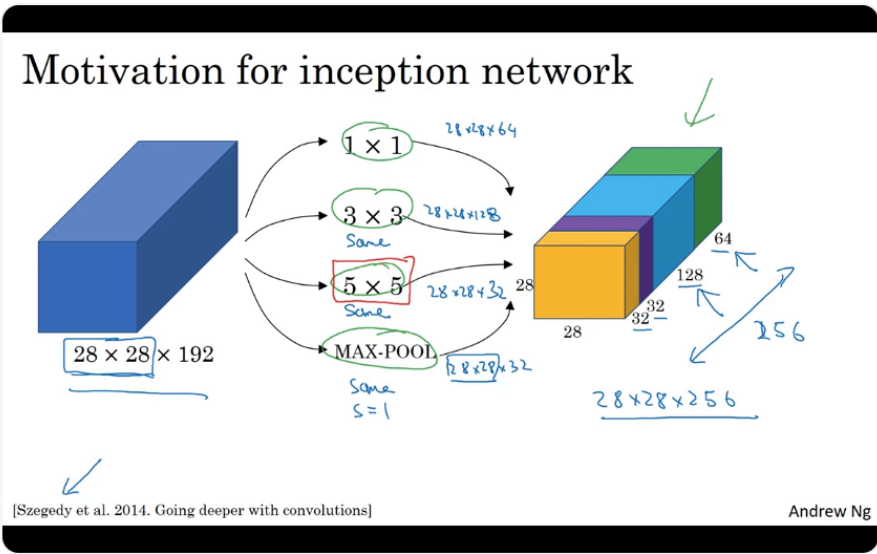
 Preparing for Inception

In the next section on **Inception Networks**, we will see these 1×1 convolutions used as "**Bottleneck Layers**." By shrinking the number of channels *before* applying expensive 3×3 or 5×5 convolutions, we can significantly speed up the network without a major loss in performance.

[!TIP]

Think of a 1×1 convolution as "**Spatial-Position Pooling**." It looks at one pixel across all layers and summarizes that information into a new set of values

Inception Network Motivation



Inception Network: The "Do It All" Layer

The Inception architecture (specifically GoogLeNet) introduces a paradigm shift: instead of choosing a single filter size (3×3, 5×5) or a pooling layer, the **Inception Module** performs them all in parallel and concatenates the results.

 The Inception Module Structure

Given an input volume (e.g., **28×28×192**), the module applies four different operations simultaneously:

1. **1×1 Convolution:** Outputs a volume of $28 \times 28 \times 64$.
2. **3×3 Convolution (Same Padding):** Outputs a volume of $28 \times 28 \times 128$.
3. **5×5 Convolution (Same Padding):** Outputs a volume of $28 \times 28 \times 32$.
4. **Max Pooling (Same Padding & Stride 1):** Outputs a volume of $28 \times 28 \times 32$.

Channel Concatenation

After these operations, the outputs are stacked along the channel dimension.

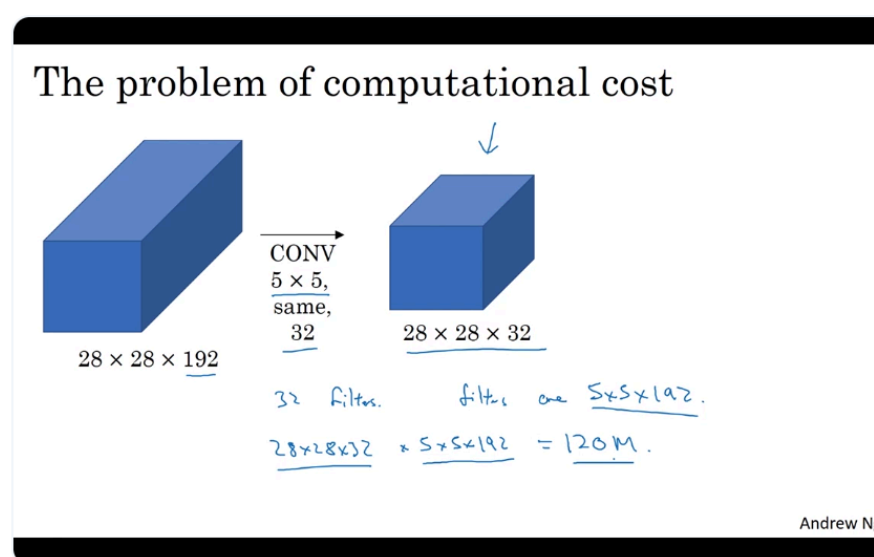
- **Calculation:** $64 + 128 + 32 + 32 = 256$
- **Final Output:** $28 \times 28 \times 256$

💡 Core Design Philosophy

- **Hedging Bets:** Instead of the designer deciding which feature size is best for a specific layer, the network "learns" which path is most useful through its weights.
- **Dimensional Consistency:** To make concatenation possible, every path must maintain the same height and width. This is achieved using **Same Padding** for all convolutions and even for the Max Pooling layer.
- **Computational Complexity:** While powerful, doing all these operations in parallel is extremely expensive—especially the 5×5 convolution.

📖 Key Facts

- **Authors:** Christian Szegedy et al. (from Google).
- **Also known as:** GoogLeNet (a play on LeNet).
- **The "Problem":** High computational cost. On a 5×5 convolution, you are performing a massive number of multiplications for every single pixel.



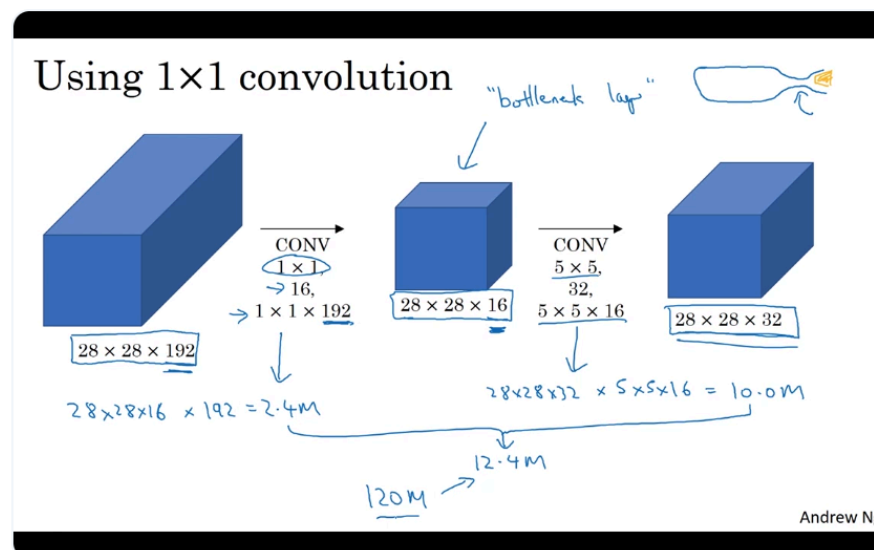
⚠️ The Computational Bottleneck

In a standard 5×5 convolution on a 192-channel input:

- Each filter is $5 \times 5 \times 192 = 4,800$ parameters.
- To produce 32 output filters, you perform $4,800 \times 32$ multiplications for **every single pixel** in the 28×28 output.
- **Total Multiplications:** $28 \times 28 \times 32 \times 5 \times 5 \times 192 \approx 120,000,000$

[!CAUTION]

120 Million operations for just one part of one layer! To fix this, Inception uses the **1×1 Convolution** as a "Bottleneck Layer" to reduce the 192 channels *before* the 5×5 convolution.



The Bottleneck Layer: Solving Computational Cost

In the previous section, we saw that a standard 5×5 convolution on a large volume could cost **120 million** multiplications. By using a **1×1 convolution** as an intermediate step, we can reduce this cost by roughly a factor of 10 without sacrificing performance.

🍷 What is a Bottleneck Layer?

A **Bottleneck Layer** is an intermediate 1×1 convolutional layer used to shrink the channel dimension before applying a more computationally expensive filter (like 3×3 or 5×5).

- **Analogy:** Like the neck of a bottle, it is the narrowest part of the network where the representation is compressed before being expanded again.

📊 Cost Comparison: Standard vs. Bottleneck

1. Standard Path (Recap)

- **Input:** $28 \times 28 \times 192$
- **Operation:** 5×5 conv, 32 filters
- **Total Cost:** ≈ 120 million multiplications.

2. Bottleneck Path (The Optimization)

We split the process into two steps:

1. **Step 1 (Shrink):** Use 16 filters of 1×1 to reduce channels from 192 to 16.
 - *Calculation:* $(28 \times 28 \times 16)$ outputs $\times (1 \times 1 \times 192)$ multiplications
 - *Cost:* **2.4 million**
2. **Step 2 (Expand):** Use 32 filters of 5×5 on the new 16-channel volume.
 - *Calculation:* $(28 \times 28 \times 32)$ outputs $\times (5 \times 5 \times 16)$ multiplications
 - *Cost:* **10.0 million**

Total Cost: $2.4\text{M} + 10.0\text{M} = 12.4$ million

Result: A reduction from 120M to 12.4M (**$\sim 10\times$ speedup**).

⚖️ Performance vs. Computation

- **Does it hurt performance?** Surprisingly, no. As long as the bottleneck isn't *too* small, the network can still pass enough information through the compressed representation to learn effectively.
- **Benefits:**
 - Drastically reduces the number of parameters and multiplications.
 - Allows the network to be deeper and more complex (like the Inception module) while remaining computationally feasible.

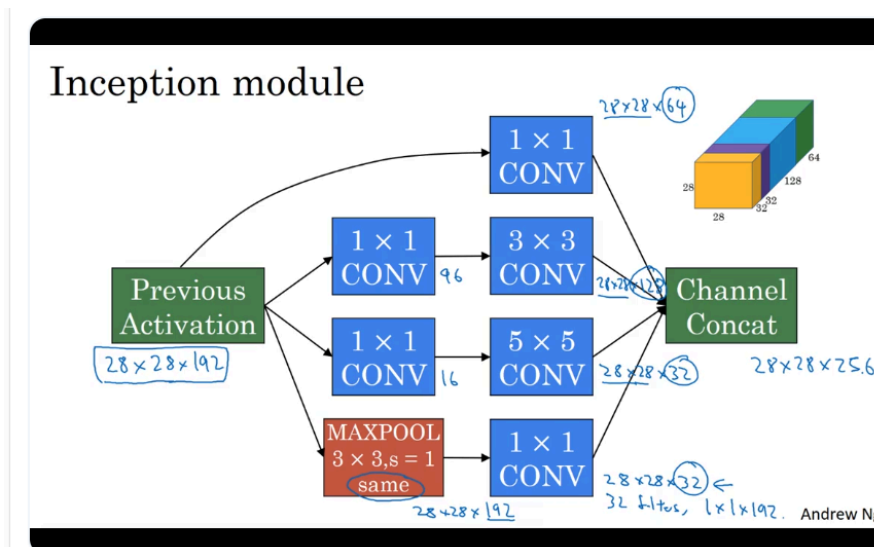
🚩 Summary of the Inception Module

- **The Problem:** Choosing the "right" filter size is hard.
- **The Inception Solution:** Apply 1×1 , 3×3 , 5×5 , and Pooling all at once and concatenate the results.
- **The Bottleneck Fix:** Use 1×1 convolutions before the 3×3 and 5×5 filters to keep the "Do It All" approach from becoming a computational nightmare.

[!TIP]

When reviewing your notes, remember: **Pooling** shrinks height/width; **1×1 Convolutions** shrink depth (channels). Together, they give you total control over the volume size!

Inception Network



Full Inception Module: Putting It All Together

An Inception network is constructed by stacking multiple **Inception Modules**. This video details how the final module is wired to balance diverse feature extraction with computational efficiency.

🧱 The Complete Inception Module

Starting with a $28 \times 28 \times 192$ input, the module splits into four parallel paths that are eventually merged.

1. The 1×1 Convolution Path

- **Operation:** A simple 1×1 convolution.
- **Purpose:** Direct feature transformation.
- **Example Output:** $28 \times 28 \times 64$.

2. The 3×3 Path (with Bottleneck)

- **Step 1:** 1×1 convolution to reduce channels (Bottleneck).
- **Step 2:** 3×3 convolution (Same padding).
- **Example Output:** $28 \times 28 \times 128$.

3. The 5×5 Path (with Bottleneck)

- **Step 1:** 1×1 convolution (Bottleneck).
- **Step 2:** 5×5 convolution (Same padding).
- **Example Output:** $28 \times 28 \times 32$.

4. The Max Pooling Path (with Post-Reduction)

- **Step 1:** Max Pooling with **Same Padding** and **Stride 1** to keep spatial dimensions at 28×28 .
- **The Problem:** Pooling alone keeps the 192 input channels, which would dominate the output.
- **Step 2:** A 1×1 convolution is applied **after** pooling to shrink the depth.

- **Example Output:** $28 \times 28 \times 32$.

Channel Concatenation

Once all four paths are calculated, they are "concatenated" along the channel dimension (depth).

- **Math:** 64 (from 1×1) + 128 (from 3×3) + 32 (from 5×5) + 32 (from Pool) = 256
- **Module Output:** $28 \times 28 \times 256$

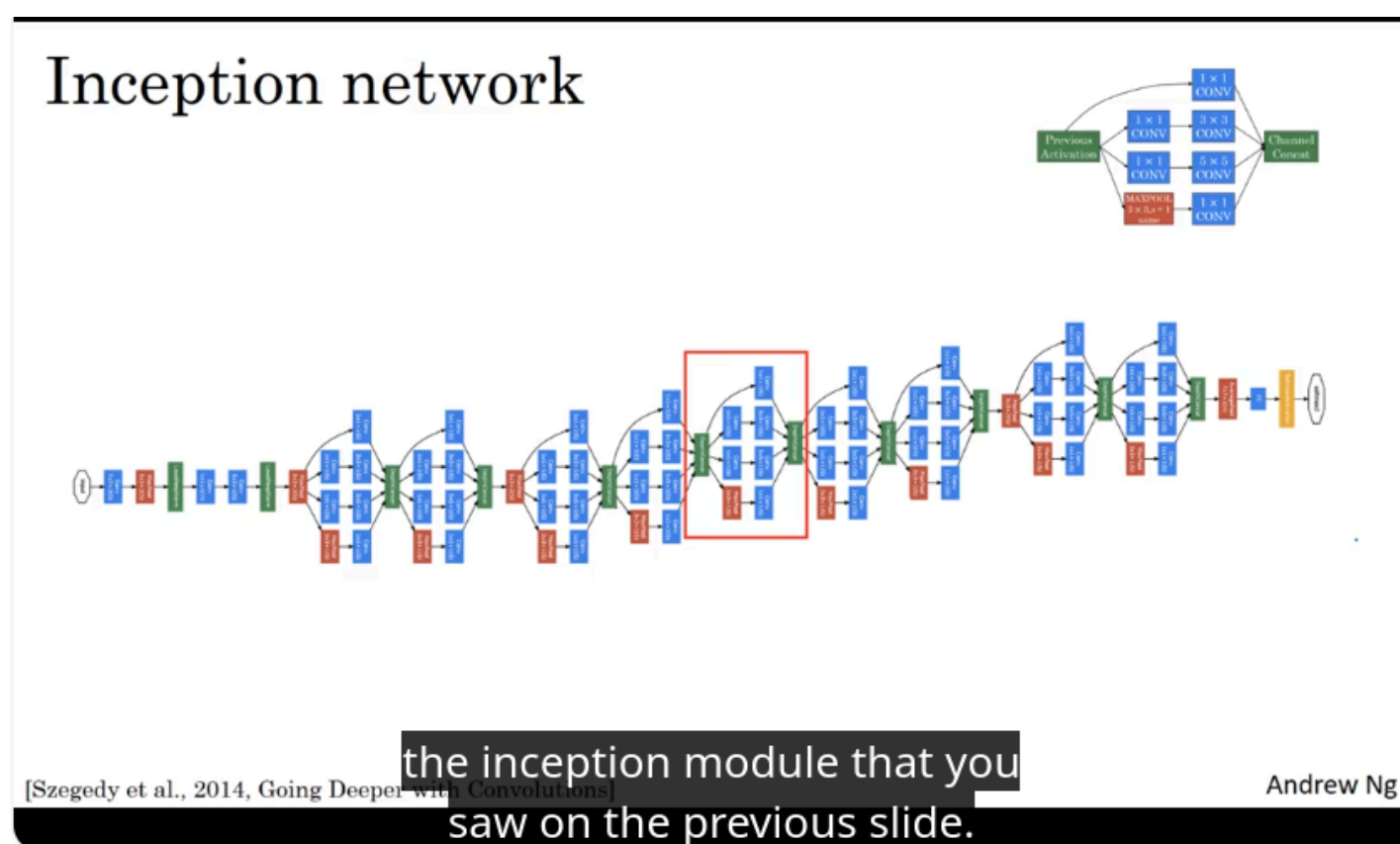
Strategic Summary

The inception module is a "network within a network" that uses:

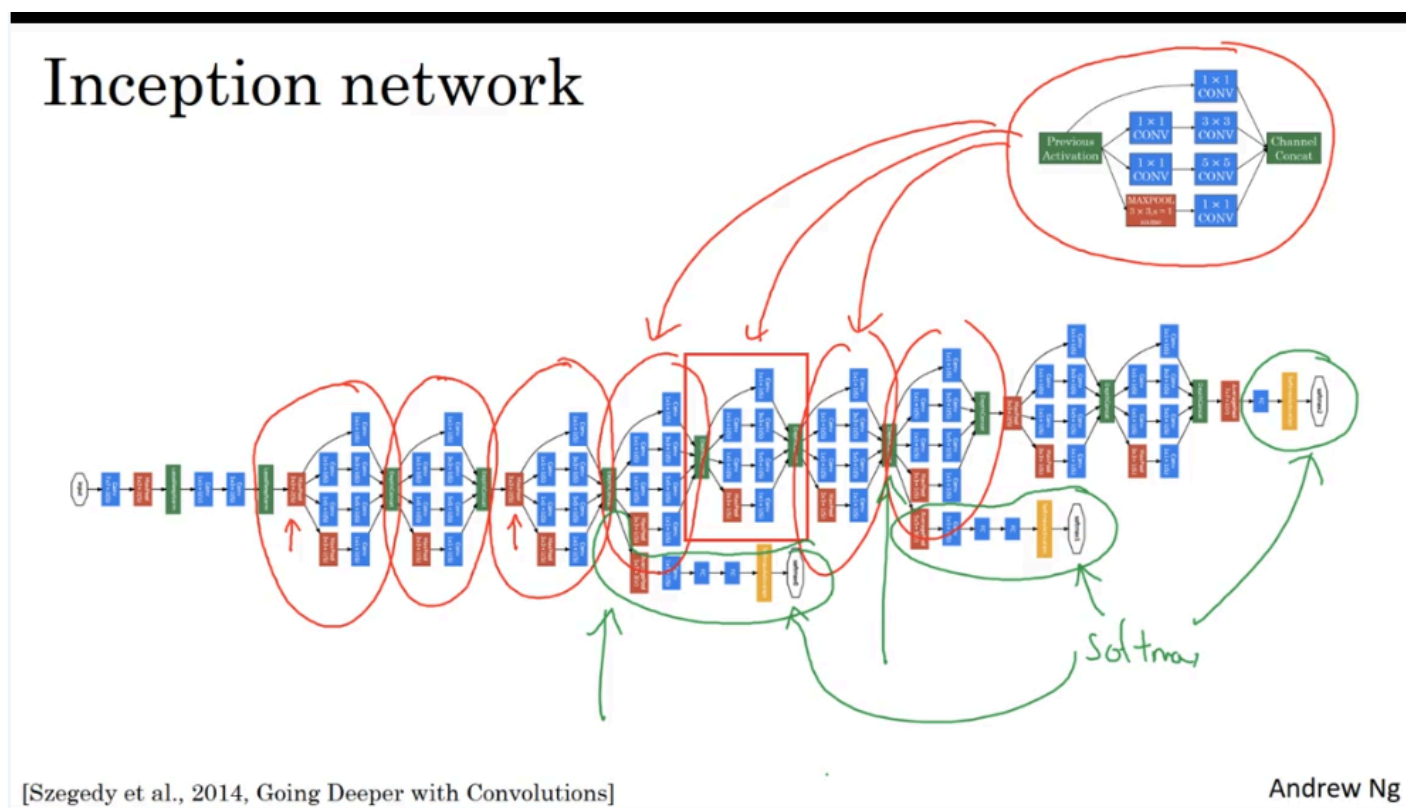
- **Parallelism:** To extract features of different scales (1×1 , 3×3 , 5×5) simultaneously.
- **Bottlenecks:** 1×1 convolutions are placed **before** expensive convolutions (3×3 , 5×5) and **after** pooling to keep the total channel count manageable.
- **Systematic Stacking:** The Inception Network (GoogLeNet) is essentially a long chain of these modules, occasionally separated by standard pooling layers to reduce spatial height and width.

[!IMPORTANT]

Key Design Choice: Notice that for pooling, the bottleneck occurs *after* the pooling operation, whereas for the 3×3 and 5×5 convolutions, the bottleneck occurs *before*. Both serve the same goal: reducing computational load and channel explosion.



Each red rectangle is an Inception Block



→ This is also called Inception network

The Full Inception Network (GoogLeNet)

The **Inception Network** (also known as **GoogLeNet**) is essentially a deep stack of the Inception modules you just learned about. While the full diagram looks intimidatingly complex, it is highly repetitive once you recognize the patterns.

Structural Layout

The network consists of a long "backbone" of layers that follow this general flow:

- **The "Stem":** A few initial standard Convolutional and Max Pooling layers to reduce the high-resolution input image.
- **The Inception Stack:** A repeated series of Inception modules.
- **Spatial Reduction:** Occasional **Max Pooling layers** placed between groups of Inception modules to halve the height and width (n_H, n_W).
- **The Output:** A Fully Connected layer followed by a Softmax to predict the final category (e.g., one of 1,000 ImageNet classes).

Auxiliary Classifiers (Side Branches)

A unique feature of the original GoogLeNet is the inclusion of **side branches** that branch off from intermediate hidden layers.

What do they do?

These branches take the features from the middle of the network and pass them through their own Fully Connected and **Softmax** layers to make a "mini-prediction" during training.

Why include them?

1. **Prevents Overfitting:** They have a **regularizing effect**, ensuring that the internal features are useful for classification early on.
2. **Combats Vanishing Gradients:** By calculating a loss in the middle of the network, they help ensure that the gradient signal remains strong even in the earlier layers.
3. **Ensures Feature Quality:** It forces the network to learn discriminative features even at intermediate stages, not just at the very end.

Fun Fact: Why "GoogLeNet"?

The authors at Google named the network **GoogLeNet** as a tribute to Yann LeCun's **LeNet-5**, which was the original pioneer of Convolutional Neural Networks.

💡 Summary for your Notion Workspace

[!NOTE]

Architecture Key:

- **Building Block:** The Inception Module (1×1, 3×3, 5×5, Pool).
- **Primary Goal:** Maximizing efficiency by "doing everything at once" while using bottlenecks to keep costs low.
- **Special Feature:** Auxiliary Classifiers (Side branches) used for regularization during training.

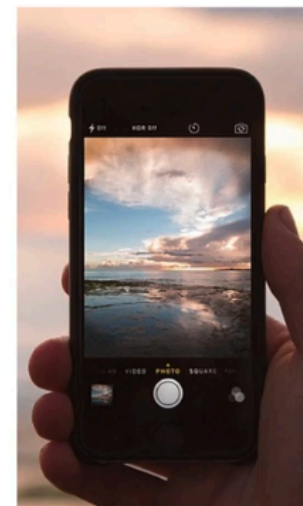
Cite of Inception Network



MobileNet

Motivation for MobileNets

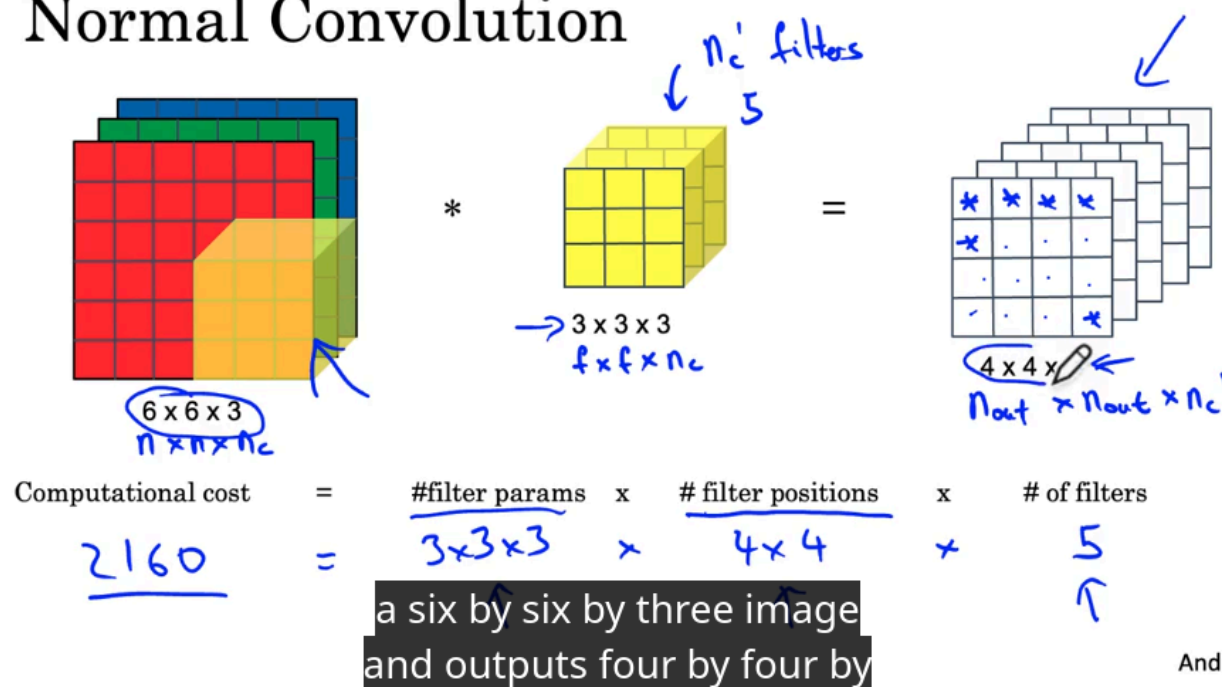
- Low computational cost at deployment
- Useful for mobile and embedded vision applications
- Key idea: Normal vs. depthwise-separable convolutions



the depthwise

[Howard et al. 2017, MobileNets: Efficient Mobile Vision Applications] Andrew Ng

Normal Convolution



Andrew Ng

MobileNets: Depthwise Separable Convolution

MobileNets are designed for **low-compute environments** (e.g., mobile phones, embedded devices) where CPUs/GPUs are less powerful. They achieve efficiency using **Depthwise Separable Convolutions**.

1. Normal Convolution (Recap)

To understand the efficiency of MobileNets, we first look at the cost of a standard convolution.

Example Setup:

- **Input Image:** $6 \times 6 \times 3$ ($n \times n \times n_C$)
- **Filter Size:** $3 \times 3 \times 3$ ($f \times f \times n_C$)
- **Number of Filters:** 5 (n'_C)
- **Output Size:** $4 \times 4 \times 5$ ($n_{out} \times n_{out} \times n'_C$)

Computational Cost Calculation:

The total number of multiplications is determined by:

$$\text{Filter Parameters} \times \text{Filter Positions} \times \text{Number of Filters}$$

- **Filter Parameters:** $3 \times 3 \times 3 = 27$
- **Filter Positions:** $4 \times 4 = 16$
- **Number of Filters:** 5
- **Total Cost:** $27 \times 16 \times 5 = 2,160$ multiplications.

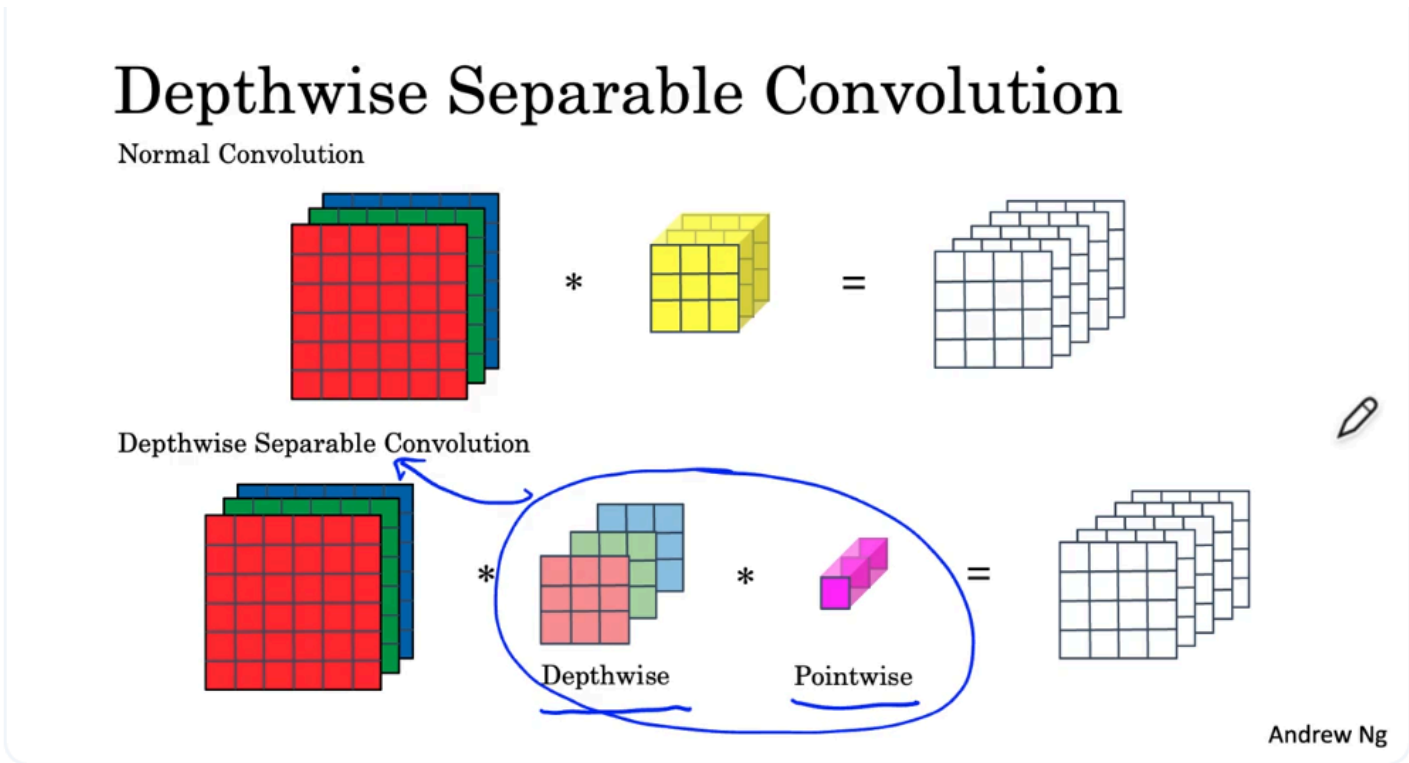
2. The Goal of MobileNet

The goal is to produce the same output dimensions ($4 \times 4 \times 5$) from the same input ($6 \times 6 \times 3$) but with a computational cost significantly lower than **2,160**.

This is achieved by breaking the normal convolution into two distinct steps:

1. **Depthwise Convolution**
2. **Pointwise Convolution**

(Note: These steps combined form the "Depthwise Separable Convolution" which will be detailed in the following section.)



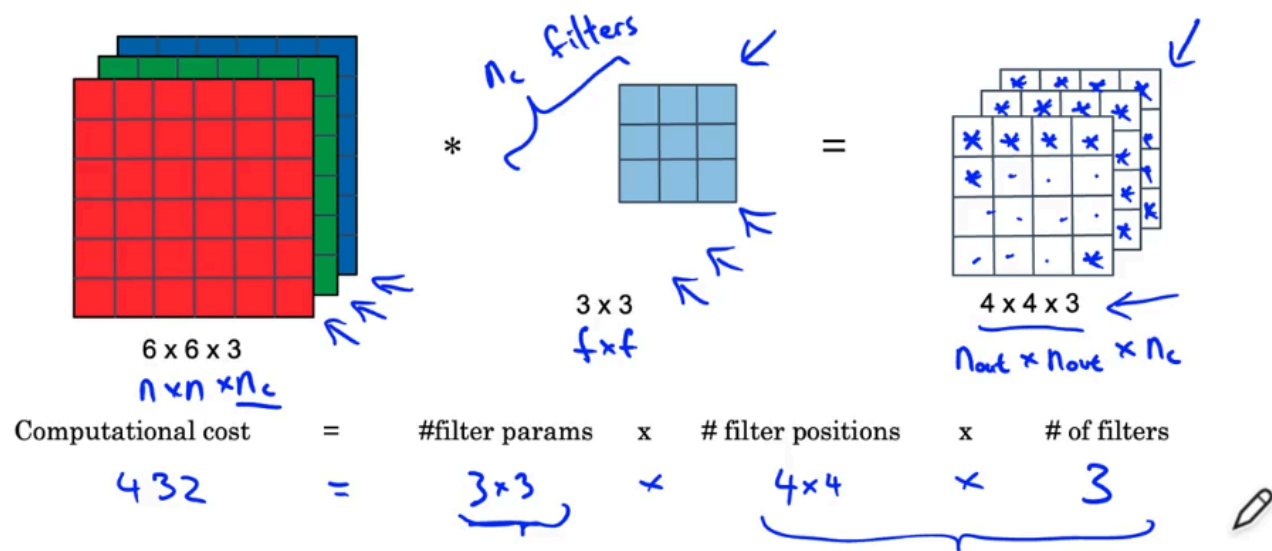
Depthwise Convolution



these filters to one of each of these input channels.

Andrew Ng

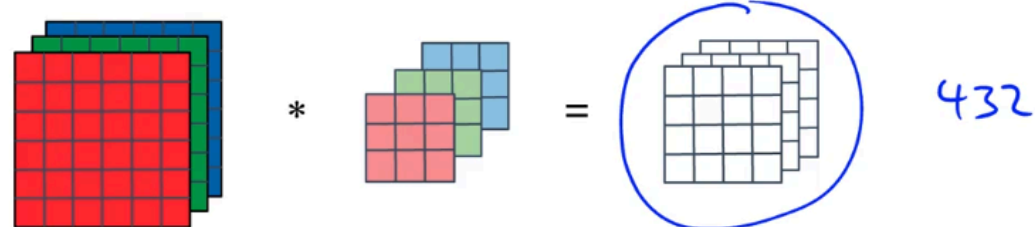
Depthwise Convolution



Andrew Ng

Depthwise Separable Convolution

Depthwise Convolution

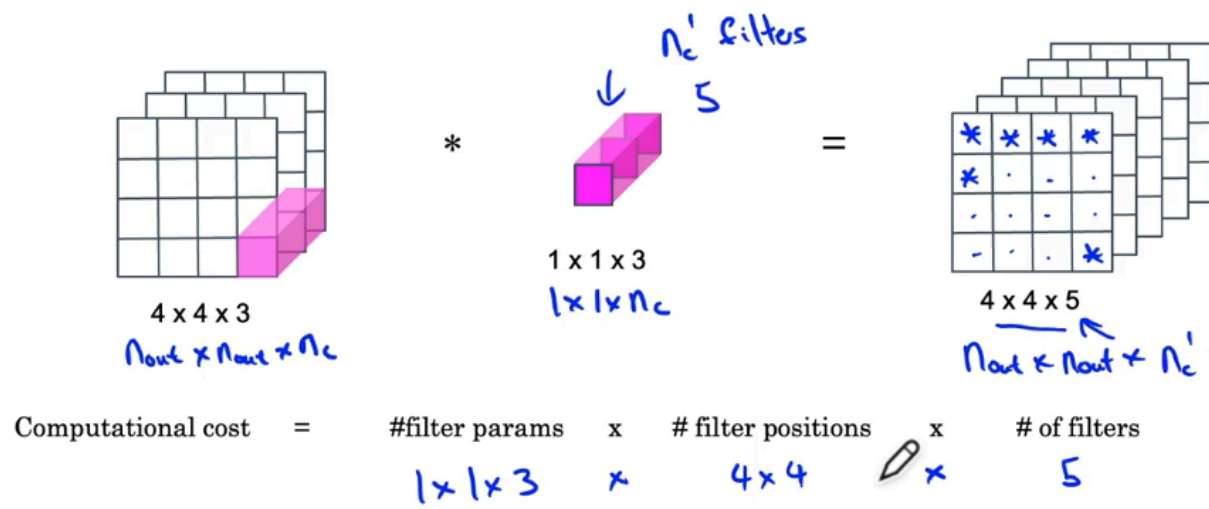


Pointwise Convolution



Andrew Ng

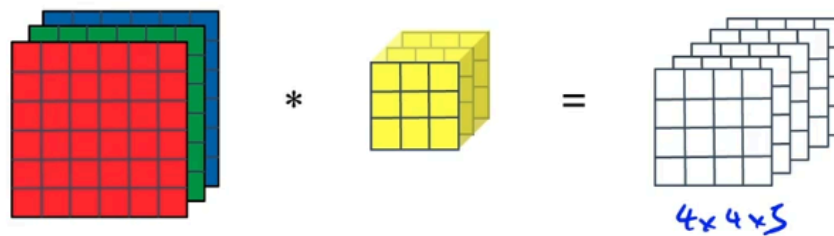
Pointwise Convolution



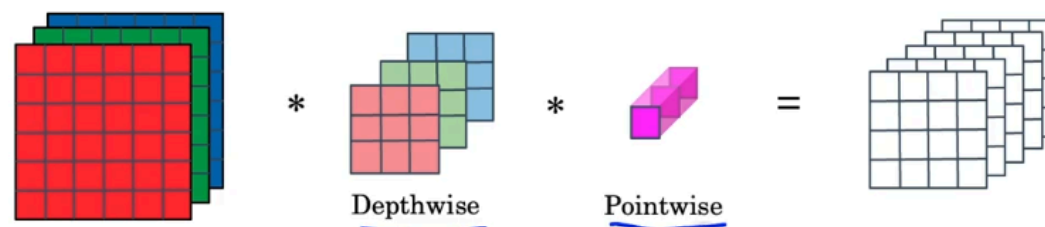
Andrew Ng

Depthwise Separable Convolution

Normal Convolution



Depthwise Separable Convolution



Andrew Ng

Cost Summary

Cost of normal convolution $\swarrow$ 2160

Cost of depthwise separable convolution $\swarrow$

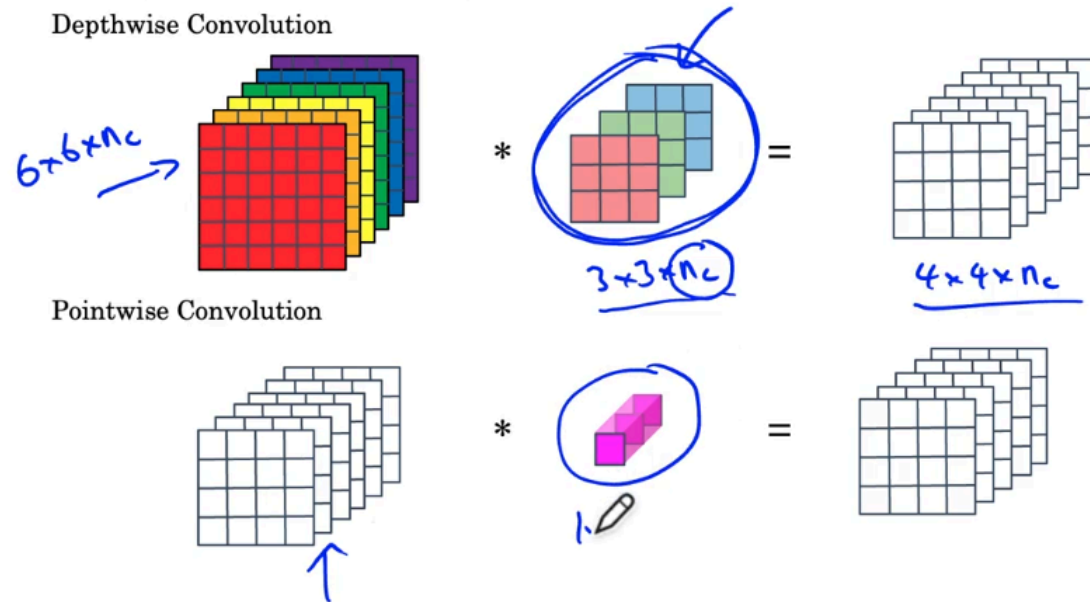
$$\begin{array}{cc} \text{depthwise} & + & \text{pointwise} \\ 432 & + & 240 = 672 \end{array}$$

$$\frac{672}{2160} = 0.31 \leftarrow$$

$$\begin{aligned} &= \frac{1}{N_c} + \frac{1}{f^2} \\ &= \frac{1}{5} + \frac{1}{9} \\ &= \frac{1}{512} + \frac{1}{3^2} \\ &\sim 10 \text{ times cheaper} \end{aligned}$$

[Howard et al. 2017, MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications] Andrew Ng

Depthwise Separable Convolution



Andrew Ng

Depthwise Separable Convolutions are the core innovation of MobileNets. They split a standard convolution into two separate stages to significantly reduce computational costs while maintaining a similar output.

The Two-Step Process

Step 1: Depthwise Convolution

In this step, we apply a single filter to **each** input channel individually.

- **Input:** $6 \times 6 \times 3$ ($n \times n \times n_c$)
- **Filters:** 3 filters of size 3×3 (one for each channel: Red, Green, Blue).
- **Operation:** Each filter slides across its specific channel to produce a 4×4 output.
- **Intermediate Output:** $4 \times 4 \times 3$ (The height and width are reduced, but the number of channels remains the same as the input).

Computational Cost:

$$\text{Filter Size}(f \times f) \times \text{Filter Positions} \times \text{Number of Channels}(n_c)$$

$$3 \times 3 \times 16 \times 3 = \mathbf{432} \text{ multiplications.}$$

Step 2: Pointwise Convolution

We then take the intermediate $4 \times 4 \times 3$ volume and apply 1×1 convolutions to change the channel depth.

- **Input:** $4 \times 4 \times 3$
- **Filters:** 5 filters of size $1 \times 1 \times 3$ ($1 \times 1 \times n_c$).
- **Operation:** Each filter looks at every pixel across all 3 channels to compute a single value.
- **Final Output:** $4 \times 4 \times 5$ ($n_{out} \times n_{out} \times n'_c$).

Computational Cost:

$$\text{Filter Size}(1 \times 1 \times n_c) \times \text{Filter Positions} \times \text{Number of Filters}(n'_c)$$

$$1 \times 1 \times 3 \times 16 \times 5 = \mathbf{240} \text{ multiplications.}$$

Efficiency Comparison

Convolution Type	Steps	Total Multiplications
Normal	1 Step	2,160
Depthwise Separable	432 (Depthwise) + 240 (Pointwise)	672



Cost Ratio: In this example, the separable convolution is $\frac{672}{2160} \approx 0.31$ (~31% of the original cost).

General Formula for Cost Savings

The authors of MobileNet show that the ratio of cost is approximately:

$$\frac{1}{n'_C} + \frac{1}{f^2}$$

- In a typical network where n'_C (output channels) is large (e.g., 512) and $f = 3$, the cost is roughly $\frac{1}{9}$.

- Key Takeaway:** Depthwise separable convolutions are roughly **10 times cheaper** than normal convolutions.

Visual Notation for MobileNet

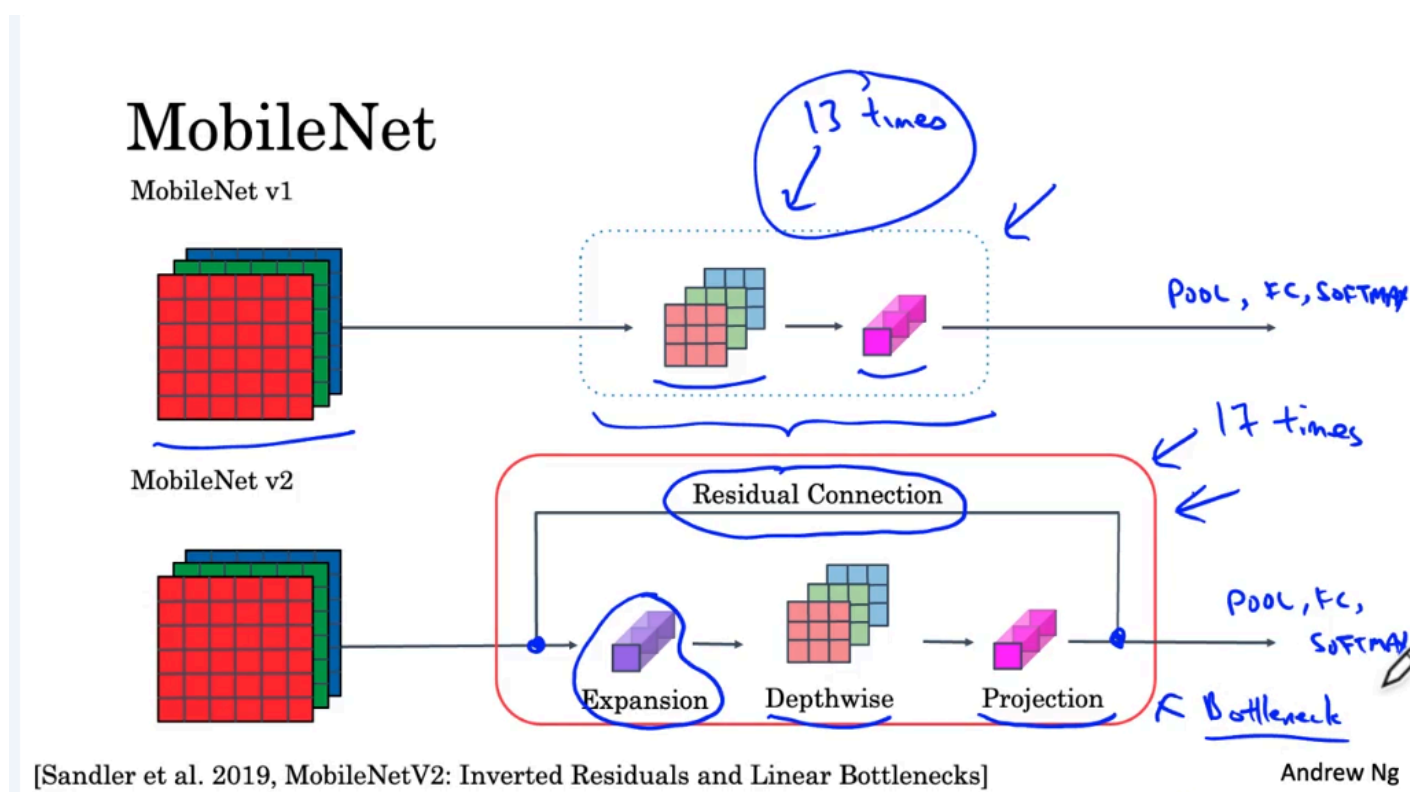
To simplify complex diagrams, MobileNet architectures often use specific icons:

- Depthwise Icon:** Represented as a stack of three filters (regardless of actual channel count).
- Pointwise Icon:** Represented as a thin pink block.

[!TIP]

Think of **Depthwise** as filtering **spatial** features (height/width) and **Pointwise** as filtering **channel** features. Together, they do the job of a normal convolution at a fraction of the price!

MobileNet Architecture



MobileNet v1 vs. MobileNet v2

Building on depthwise separable convolutions, the MobileNet family evolved to become even more efficient and capable of training deeper structures.

📱 MobileNet v1 Architecture

The core philosophy of v1 is to replace every expensive standard convolution with a **depthwise separable convolution** block.

- **Structure:** It consists of a stack of **13** depthwise separable convolution blocks.
- **The Block:** Each block contains one **Depthwise Convolution** followed by a **Pointwise Convolution**.
- **Final Layers:** After the 13 blocks, the network uses standard Global Average Pooling, a Fully Connected layer, and a Softmax for classification.

🚀 MobileNet v2: The Bottleneck Block

MobileNet v2 (by Sandler et al.) introduced two significant architectural improvements to make the network even more efficient and easier to train.

1. Residual Connections

Just like in ResNet, v2 incorporates **skip connections**. These allow gradients to flow more easily through the 17 blocks of the network, helping with the training of deeper architectures.

2. Expansion Layer and Projection

While v1 went straight to the depthwise step, v2 uses a three-step process within its "Bottleneck Block":

1. **Expansion Layer (1×1 Conv):** It first increases the number of channels (expanding the representation). This gives the depthwise convolution more features to work with.
2. **Depthwise Convolution:** Performs the spatial filtering on the expanded volume.
3. **Projection Layer (1×1 Conv):** Also known as a pointwise convolution, this step "projects" the high-dimensional data **back down to a smaller number of channels** to keep the computational cost low for the next block.

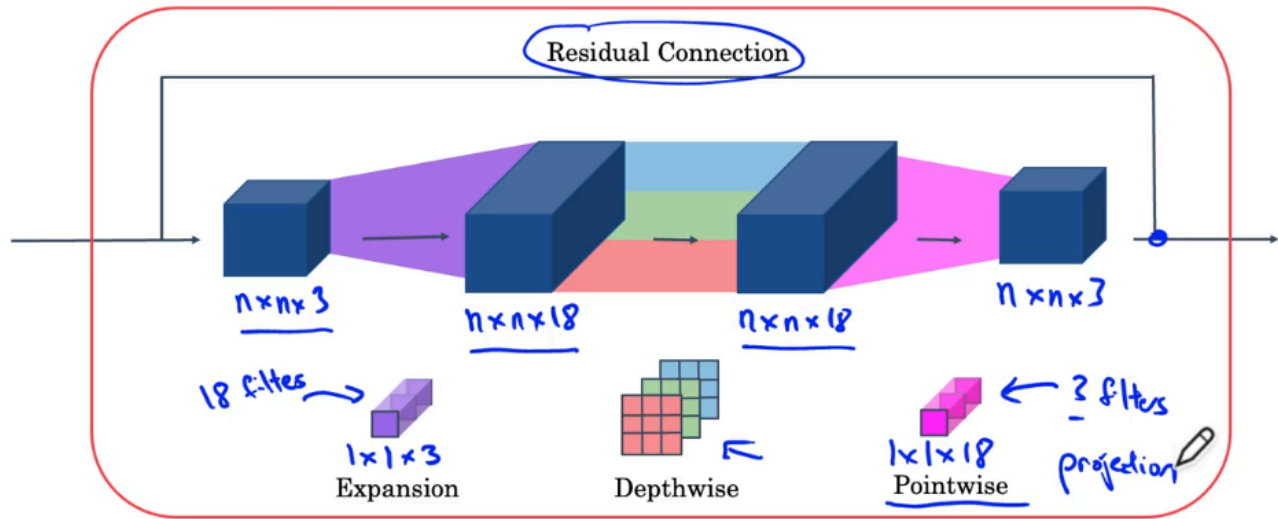
📊 Key Differences at a Glance

Feature	MobileNet v1	MobileNet v2
Primary Block	Depthwise Separable Conv	Bottleneck Block
Main Layers	13 Blocks	17 Blocks
Skip Connections	No	Yes (Residual connections)
Internal Steps	2 steps (Depthwise → Pointwise)	3 steps (Expansion → Depthwise → Projection)

[!NOTE]

The term **"Bottleneck"** is used for the v2 block because the input and output of the block have relatively few channels compared to the "expanded" internal representation.

MobileNet v2 Bottleneck



[Sandler et al. 2019, MobileNetV2: Inverted Residuals and Linear Bottlenecks]

Andrew Ng

MobileNet v2 Bottleneck: Deep Dive

MobileNet v2 improves upon v1 by using the **Bottleneck Block**, which balances the need for high-performance learning with the strict memory constraints of mobile devices.

Inside the Bottleneck Block

The block processes an input (e.g., $n \times n \times 3$) through three distinct stages:

1. Expansion (1x1 Convolution)

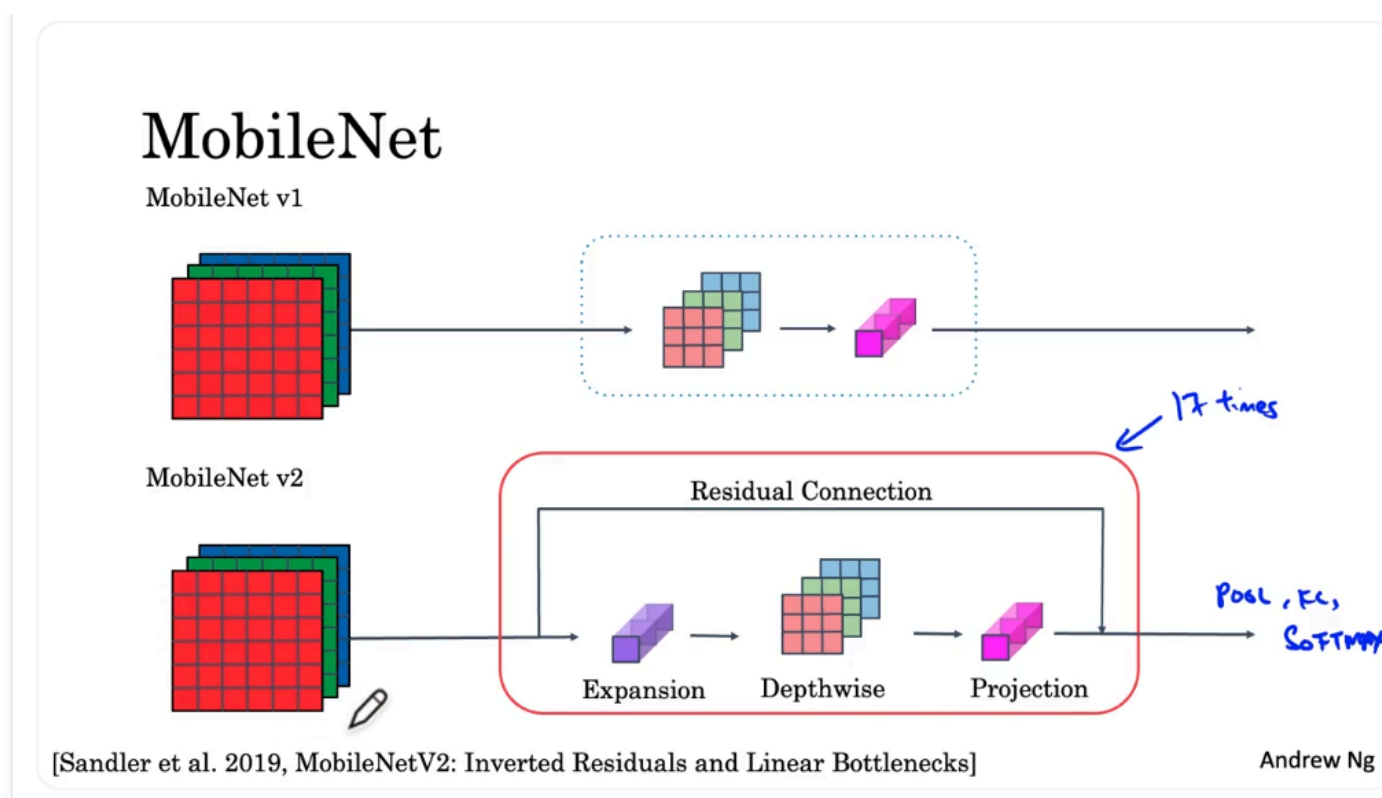
- **Operation:** Applies a 1×1 convolution with many filters.
- **Typical Expansion Factor: 6x.**
- **Result:** Converts $n \times n \times 3$ into a much larger $n \times n \times 18$ volume.
- **Purpose:** Increases the representation size, allowing the network to learn much richer and more complex functions.

2. Depthwise Convolution

- **Operation:** Standard 3×3 depthwise convolution.
- **Padding:** Uses padding to ensure spatial dimensions remain $n \times n$
- **Result:** Filters the expanded $n \times n \times 18$ volume without changing its size.

3. Projection (Pointwise Convolution)

- **Operation:** Applies a 1×1 convolution with a small number of filters (e.g., 3 filters).
- **Result:** Projects the data back down from $n \times n \times 18$ to $n \times n \times 3$
- **Purpose:** Shrinks the activation size to save memory. When passing data between blocks, the device only needs to store the smaller "bottlenecked" dimensions.



The Residual Connection

Just like in ResNets, the bottleneck block uses a **skip connection** that passes the input directly to the output.

- **Function:** $Output = Expansion_Path(Input) + Input$
- **Benefit:** Helps gradients propagate backward efficiently during training.

Why use this "Expand-then-Shrink" Strategy?

MobileNet v2 provides a clever trade-off:

- **Rich Computation:** The "expanded" middle section provides the high-dimensional space needed for complex learning.
- **Memory Efficiency:** The "projected" ends ensure that the memory footprint required to store activations between layers remains small.

🏗️ Full Network Structure

- **Repetition:** The bottleneck block is repeated **17 times**.
- **Final Stage:** Ends with Global Average Pooling → Fully Connected → Softmax.
- **Performance:** Achieves better accuracy than MobileNet v1 while staying within modest compute and memory limits.

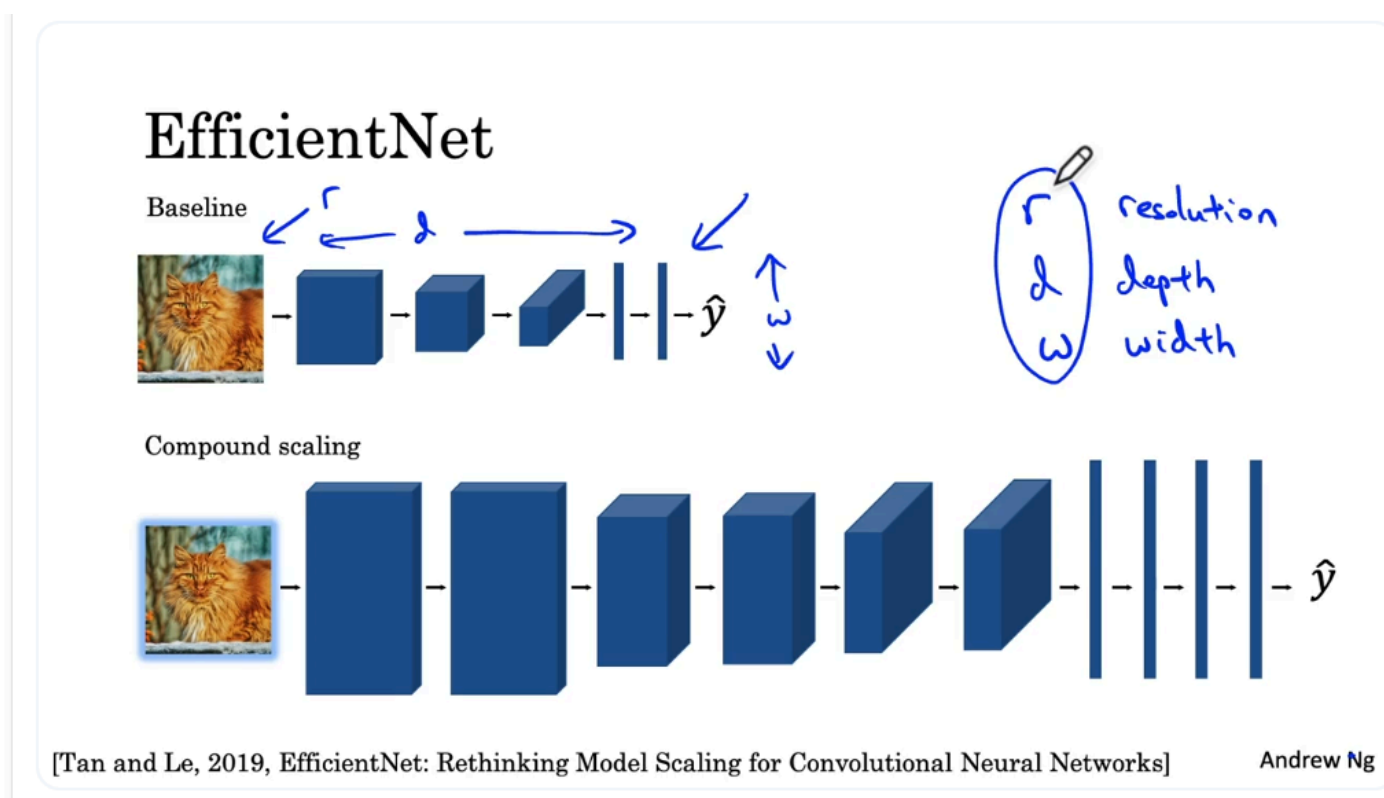
[!TIP]

Summary:

1. **Input:** Small depth (saves memory).
2. **Expansion:** Huge depth (learns complex features).
3. **Depthwise:** Spatial filtering.
4. **Projection:** Small depth (prepares for the next block).

Next Steps for your Notion: You have now covered the evolution from standard convolutions to the highly efficient MobileNet v2. The next concept mentioned is **EfficientNet**, which focuses on scaling these architectures optimally.

EfficientNet



EfficientNet: Compound Scaling

While MobileNets focus on efficient building blocks, **EfficientNet** focuses on how to scale those architectures optimally for different devices. Whether you have a high-end smartphone or a low-compute edge device, EfficientNet helps you find the best balance between accuracy and speed.

📏 The Three Scaling Dimensions

The authors (Mingxing Tan and Quoc Le) identified three primary ways to scale a convolutional neural network:

1. **Resolution (r):** Using higher-resolution input images. High-res images provide more fine-grained patterns but require more computation.
2. **Depth (d):** Adding more layers to the network. Deeper networks can learn more complex features but are harder to train and slower.
3. **Width (w):** Increasing the number of channels/filters in each layer. Wider networks can capture more fine-grained features but quickly become computationally expensive.

Compound Scaling

The core "aha!" moment of EfficientNet is that scaling these three dimensions (r, d, w) independently is sub-optimal. For example, if you increase the image resolution, you should also increase the depth and width so the network has enough capacity and receptive field to process the extra pixels.

Compound Scaling uses a single compound coefficient to simultaneously scale up or down all three dimensions at a fixed, optimal rate.

[!IMPORTANT]

The Trade-off: EfficientNet provides a systematic way to say: "If I have $2\times$ more compute, I should increase depth by α , width by β , and resolution by γ to get the maximum possible accuracy gain."

Why use EfficientNet?

- **Adaptability:** It allows you to automatically scale a baseline network (like EfficientNet-B0) to larger versions (B1 through B7) depending on your hardware's power.
- **Efficiency:** It often achieves higher accuracy than previous "human-designed" architectures while using significantly fewer parameters and operations (FLOPs).
- **Open Source:** Most deep learning frameworks have open-source implementations of EfficientNet that handle these trade-offs for you automatically.

Summary of Architectures

Architecture	Main Contribution	Best For...
MobileNet v1/v2	Depthwise Separable Convolutions & Bottlenecks	Mobile & Edge devices (Fixed efficiency).
EfficientNet	Compound Scaling (r, d, w)	Optimizing performance for <i>any</i> specific compute budget.

Expert Guide Follow-up

You have now completed the core survey of modern CNN architectures! Since you're building this in Notion, would you like a final **Architecture Selection Tree** that helps you decide which of these models to use based on a specific project's needs (e.g., speed vs. accuracy vs. memory)?